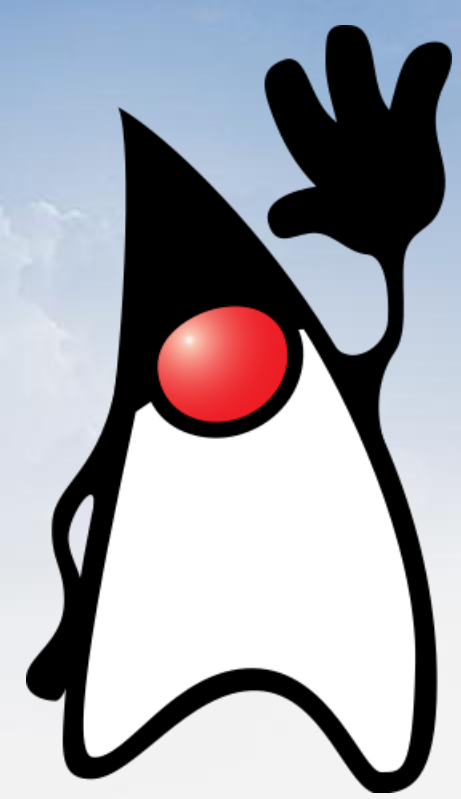




Polymorphism

COP 3330
Department of Computer Science
University of Central Florida
Dr. Bacanli

Why do we need to know this?

- You have just learned inheritance and how certain methods between child and parent classes are called.
- It is important to know the details of why this can happen and what some limitations are.

Binding

```
public class Obj1{  
    void method1(){ .... }  
    void method2(){ ....}  
  
    public static void main(String[] args){  
        Obj1 anObj=new Obj1();  
        anObj.method1();  
        anObj.method2();  
    }  
}
```



Identifying for a particular method call, which method is being executed is defined as binding

What is static and dynamic binding?

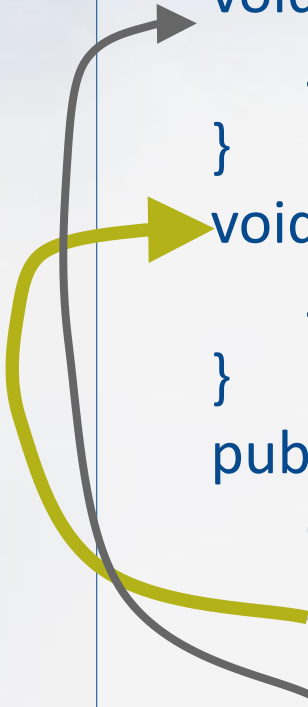
- Binding – connecting some call to a method to a particular definition.
- Static Binding – The type of object is determined a compile time.
- Dynamic Binding – The type of object is determined during run time.

Static Binding

- The type of object is determined at compile time.
- If static, final, or private is used in a method, then static binding will take place.
- Compiler knows which method will be executed before running.
- Method overloading is an example.

Static Binding Example

```
public class Obj1{  
    void method(int num1,int num2){  
        System.out.println(num1+num2);  
    }  
    void method(int num1){  
        System.out.println( (num1*2+7) );  
    }  
    public static void main(String[] args){  
        Obj1 anObj=new Obj1();  
        anObj.method(5);  
        anObj.method(5,4);  
    }  
}
```



Before we run compiler knows(in compile time) which method to be invoked. HOW?

Dynamic(Late) Binding

- The type of object is determined at run time.
- We have been seeing this when we were introduced to inheritance.
- The main thing you will want to observe is the Object Reference!
- Compiler cannot know which function to run before running.
- Method overriding is an example.

Dynamic Binding Example

```
class Car{  
    void run(){  
        System.out.println("Car is running");  
    }  
}
```

```
class Volvo extends Car{  
    void run(){  
        System.out.println("Volvo is running");  
    }  
}
```

```
class Lexus extends Car{  
    void run(){  
        System.out.println("Lexus is running");  
    }  
}
```

```
public class Test{  
    public static void main(String[] args){  
        Car c;  
        c = new Car();  
        c.run();  
        c = new Volvo();  
        c.run();  
        c = new Lexus();  
        c.run();  
    }  
}
```

Makes the binding based on the object type not reference variable

Static vs Dynamic Binding

Static Binding	Dynamic Binding
Happens during compile time	Happens during run time
Reference type is used to bind the method	Actual object type is used to bind the method.
Early Binding	Late Binding
Very Fast	Very Slow
Overloading is Resolved	Overriding is Resolved

Car example

```
public class Vehicle{  
    public int wheelNum; // by default=0  
  
    public void go(){  
        System.out.println("Vehicle is going");  
    }  
}
```

```
public class Car extends Vehicle{  
    public int vinNumber=117;  
  
    public void drift(){  
        System.out.println("Car is drifting");  
    }  
}
```

go() method will work for Vehicle and Car.
wheelNum is accessible for Vehicle and Car.

Can drift method work for Vehicle?

```
Vehicle tractor=new Vehicle();  
tractor.drift();
```

If those make sense, let us continue



Upcasting

```
Car realCar = new Car();  
Vehicle castedCar = new Car();
```

We do this to make our objects more flexible for our implementations
However with castedCar,
You can only call the overridden methods of Car but not the actual
methods of Car

```
realCar.go(); // this will work  
castedCar.drift(); //this wont work, Vehicle has no drift defined  
castedCar.go(); // overridden method will be run, the method in Car  
System.out.println(castedCar.wheelNum); // will print 0
```



Downcasting

```
Car realCar = new Car();  
Vehicle castedCar = new Car();  
Car downcasted= (Car)castedCar;
```

We do this to narrow down a generalization.

This means “Hey Java I know this is a Car. Please behave accordingly”.

```
realCar.go(); // this will work  
castedCar.drift(); //this wont work  
((Car)castedCar).drift(); // this will work  
downcasted.drift(); //this will work  
System.out.println( ((Car)castedCar).vinNumber); //this will work
```

Code Example

```
package carEx;
public class Vehicle{
    private int wheelNumber;

    public Vehicle(int w){
        wheelNumber = w;
    }

    public void go() {
        System.out.println("vehicle go on "
            +wheelNumber);
    }
}
```

vehicle go on 4
ford drift

```
package carEx;
public class Car extends Vehicle{
    private String brandName;

    public Car(String bname){
        super(4);
        brandName = bname;
    }
    public void drift() {
        System.out.println(brandName + " drift");
    }
}
```

```
package carEx;
public class VehiclesTester{
    public static void main(String[] args){
        Vehicle castedcar = new Car("ford");
        castedcar.go(); // inherited method call
        ((Car)castedcar).drift(); // Car method
        // without paranthesis, compilation error
    }
}
```

Which method to write/run?

```
Vehicle v = new Car();  
v.run();
```

What we can write after `.` is decided by the reference. (Vehicle)
we can only write the methods implemented inside Vehicle

What method will be run for `v.run()`; statement is decided by object (Car)
if run is overridden, overridden run() will be called.
if not overridden, the inherited run() method will be called

```

public class Vehicle {
    public void go() {
        System.out.println("Vehicle going");
    }
}

public class Car extends Vehicle {
    public void go() {
        System.out.println("car goes");
    }
}

```

The object reference decides which method(Vehicle or Car one) to run.

The good news is the overridden methods will still work as expected!

```

public class vtester{
    public static void method(Car a) {
        System.out.print("Car param method "); a.go();
    }
    public static void method(Vehicle a) {
        System.out.print("Vehicle param method "); a.go();
    }
    public static void methodVehicle(Vehicle a) {
        System.out.print("methodVehicle "); a.go();
    }

    public static void main(String[] args){
        Car realCar = new Car();
        Vehicle castedCar = new Car();
        System.out.println("** RealCar **");
        method(realCar);
        method((Vehicle)realCar);
        methodVehicle(realCar);
        System.out.println("** CastedCar **");
        method((Car)castedCar);
        method(castedCar);
        methodVehicle(castedCar);
    } //end of main
}

```

```

public class Vehicle {
    public void go() {
        System.out.println("Vehicle going");
    }
}

public class Car extends Vehicle {
    public void go() {
        System.out.println("car goes");
    }
}

```

```

** RealCar **
Car param method car goes
Vehicle param method car goes
methodVehicle car goes
** CastedCar **
Car param method car goes
Vehicle param method car goes
methodVehicle car goes

```

Note that with `castedCar`, we can't run `car` param method without down casting again to `Car`! We never see *vehicle going*!

```

public class vtester{
    public static void method(Car a) {
        System.out.print("Car param method "); a.go();
    }
    public static void method(Vehicle a) {
        System.out.print("Vehicle param method "); a.go();
    }
    public static void methodVehicle(Vehicle a) {
        System.out.print("methodVehicle "); a.go();
    }

    public static void main(String[] args){
        Car realCar = new Car();
        Vehicle castedCar = new Car();
        System.out.println("** RealCar **");
        method(realCar);
        method((Vehicle)realCar);
        methodVehicle(realCar);
        System.out.println("** CastedCar **");
        method((Car)castedCar);
        method(castedCar);
        methodVehicle(castedCar);
    } //end of main
}

```

Polymorphism

```
public class MathOperations {  
    public int add(int a, int b) {  
        return a + b;  
    }  
  
    public double add(double a, double b)  
    {  
        return a + b;  
    }  
}
```

- Compile-Time Polymorphism (Static Polymorphism):
 - This is achieved through method overloading, where multiple methods in the same class have the same name but different parameter lists.
 - The decision about which method to call is made at compile time based on the method signature.

Polymorphism

```
class Animal {  
    void makeSound() {  
        System.out.println("Some sound");  
    }  
}  
  
class Dog extends Animal {  
    @Override  
    void makeSound() {  
        System.out.println("Woof!");  
    }  
}
```

- Runtime Polymorphism (Dynamic Polymorphism)
 - This is achieved through method overriding, where a subclass provides a specific implementation for a method that is already defined in its superclass.
 - The decision about which method to call is made at runtime based on the actual type of the object.

Polymorphism with Interfaces

```
public interface InterestGiving{
    public abstract void writeInterestRate();
}

class BCE implements InterestGiving{
    public void writeInterestRate(){
        System.out.println("BCE interest is " + 13.2);
    }
}

class UHN implements InterestGiving{
    public void writeInterestRate(){
        System.out.println("UHN interest is " + 10.5);
    }
}

class CCC implements InterestGiving{
    public void writeInterestRate(){
        System.out.println("CCC interest is " + 9.8);
    }
}
```

```
class TestPolymorphism{

    public static void main(String args[]){
        InterestGiving bce = new BCE();
        InterestGiving uhn = new UHN();
        InterestGiving ccc = new CCC();
        ArrayList<InterestGiving> arr=new ArrayList<>();
        arr.add(bce);
        arr.add(uhn);
        arr.add(ccc);
        for(int i=0; i< arr.size(); i++){
            arr.get(i).writeInterestRate();
        } //end of for

    } //end of main
}
```

```
BCE interest is 13.2
UHN interest is 10.5
CCC interest is 9.8
```

Polymorphism with Inheritance

```
public abstract class Bank{
    public abstract void writeInterestRate();
}

class BCE extends Bank{
    public void writeInterestRate(){
        System.out.println("BCE interest is " + 13.2);
    } public void test(){}
}

class UHN extends Bank{
    public void writeInterestRate(){
        System.out.println("UHN interest is " + 10.5);
    }
}

class CCC extends Bank{
    public void writeInterestRate(){
        System.out.println("CCC interest is " + 9.8);
    }
}
```

```
class TestPolymorphism{

    public static void main(String args[]){
        Bank bce = new BCE();
        Bank uhn = new UHN();
        Bank ccc = new CCC();
        ArrayList<Bank> arr=new ArrayList<>();
        arr.add(bce);
        arr.add(uhn);
        arr.add(ccc);
        for(int i=0; i< arr.size(); i++){
            arr.get(i).writeInterestRate();
        } //end of for

    } //end of main
}
```

```
BCE interest is 13.2
UHN interest is 10.5
CCC interest is 9.8
```

Why do I need upcasting in practice?

- If you will only use the **overridden** methods of a superclass, for code **flexibility**, upcasting might be nice.

We need the return type of this method to be List. As long as it is List (ArrayList or LinkedList), we are ok to use it. You can implement this function using any List object instance. With mylist I will only call the methods which are stated in List interface.

Since the List is created as ArrayList casting is possible here.

```
package listTester;
import java.util.*;

public class Tester1 {
    public static List<Integer> getFilledList() {
        List<Integer> arr;
        arr = new ArrayList<Integer>();
        arr.add(1);
        arr.add(2);
        return arr;
    }

    public static void main(String[] args) {
        List<Integer> mylist;
        mylist = getFilledList(); // this is a list!
        System.out.println(mylist.get(0));
        ArrayList<Integer> marrlist; // this is arraylist
        marrlist = (ArrayList<Integer>)(mylist);
        System.out.println(marrlist.get(0));
    }
}
```

List<> Interface

Modifier and Type	Method and Description
boolean	add(E e) Appends the specified element to the end of this list (optional operation).
void	add(int index, E element) Inserts the specified element at the specified position in this list (optional operation).
boolean	addAll(Collection<? extends E> c) Appends all of the elements in the specified collection to the end of this list, in the order that they are returned by the specified collection's iterator (optional operation).
boolean	addAll(int index, Collection<? extends E> c) Inserts all of the elements in the specified collection into this list at the specified position (optional operation).
void	clear() Removes all of the elements from this list (optional operation).
boolean	contains(Object o) Returns true if this list contains the specified element.
boolean	containsAll(Collection<?> c) Returns true if this list contains all of the elements of the specified collection.
boolean	equals(Object o) Compares the specified object with this list for equality.
E	get(int index) Returns the element at the specified position in this list.

<https://docs.oracle.com/javase/8/docs/api/java/util/List.html>

Casting

- Arrays class has a method to populate a list using an array.
- This method returns a list, not ArrayList.
- Can we cast the List to ArrayList?
 - Casting will not solve every problem.
 - Although ArrayList is a List, we cannot cast any List to ArrayList.
 - mylist is not ArrayList object

This will give a run time Exception.
Casting will not solve this problem.
arrlist = (ArrayList)(mylist);
is also wrong.

```
package listTester;
import java.util.ArrayList;
import java.util.Arrays;
import java.util.List;

public class Tester1 {
    public static List<Integer> getFilledList(int n) {
        Integer[] num_arr=new Integer[n];
        for(int i=0;i<n;i++){
            num_arr[i]=i+1;//int array is not possible
        }
        List<Integer> list = Arrays.asList(num_arr);
        return list;
    }
    public static void main(String[] args) {
        List<Integer> mylist = getFilledList(5);
        System.out.println(mylist.get(0));

        ArrayList<Integer> arrlist;
        arrlist = (ArrayList<Integer>)(mylist);
        System.out.println(arrlist);
    }
}
```

Populate list with an array

- We cannot just cast List to ArrayList anywhere in the code.
- We will use the constructor of ArrayList to populate it with a List object.

We will populate the ArrayList using a List.

```
1  
[1, 2, 3, 4, 5]
```

```
package listTester;  
import java.util.ArrayList;  
import java.util.Arrays;  
import java.util.List;  
  
public class Tester1 {  
    public static List<Integer> getFilledList(int n) {  
        Integer[] num_arr=new Integer[n];  
        for(int i=0;i<n;i++){  
            num_arr[i]=i+1;//int array is not possible  
        }  
        List<Integer> list = Arrays.asList(num_arr);  
        return list;  
    }  
    public static void main(String[] args) {  
        List<Integer> mylist = getFilledList(5);  
        System.out.println(mylist.get(0));  
  
        ArrayList<Integer> arrlist;  
        arrlist = new ArrayList<>(mylist);  
        System.out.println(arrlist);  
    }  
}
```

Method Hiding

- Remember method overloading and method overriding?
- Method hiding can be defined as a child class defining some static method with the same signature as the **static** method in the parent class,
 - The method in the child class hides the one in the parent class
 - The parent method still can be called using parent class name.
 - It looks like child is overriding the method but parent method is still runnable.
- Static method is called based on the reference not object type.

Method hiding

```
public class Parent {  
    public static void show() {  
        System.out.println("Parent show");  
    }  
}  
  
public class Child extends Parent {  
    // Static method in the child class  
    // hiding the parent's static method  
    public static void show() {  
        System.out.println("Child show");  
    }  
}
```

```
Parent show  
Parent show  
Child show  
Child show  
Parent show
```

```
public class MethodHidingExample {  
    public static void main(String[] args) {  
        // Static method is called based on the  
        // reference type, not the actual object type.  
  
        Parent parent = new Parent();  
  
        // Parent reference to Child object  
        Parent childAsParent = new Child();  
  
        Child child = new Child();  
  
        // Calls Parent's show() method  
        parent.show();  
  
        // Calls Parent's show() method  
        childAsParent.show();  
  
        // Call Child's show() method  
        child.show();  
        Child.show(); // Calling method using Class Name  
        Parent.show(); // Calling method using Class Name  
    }  
}
```